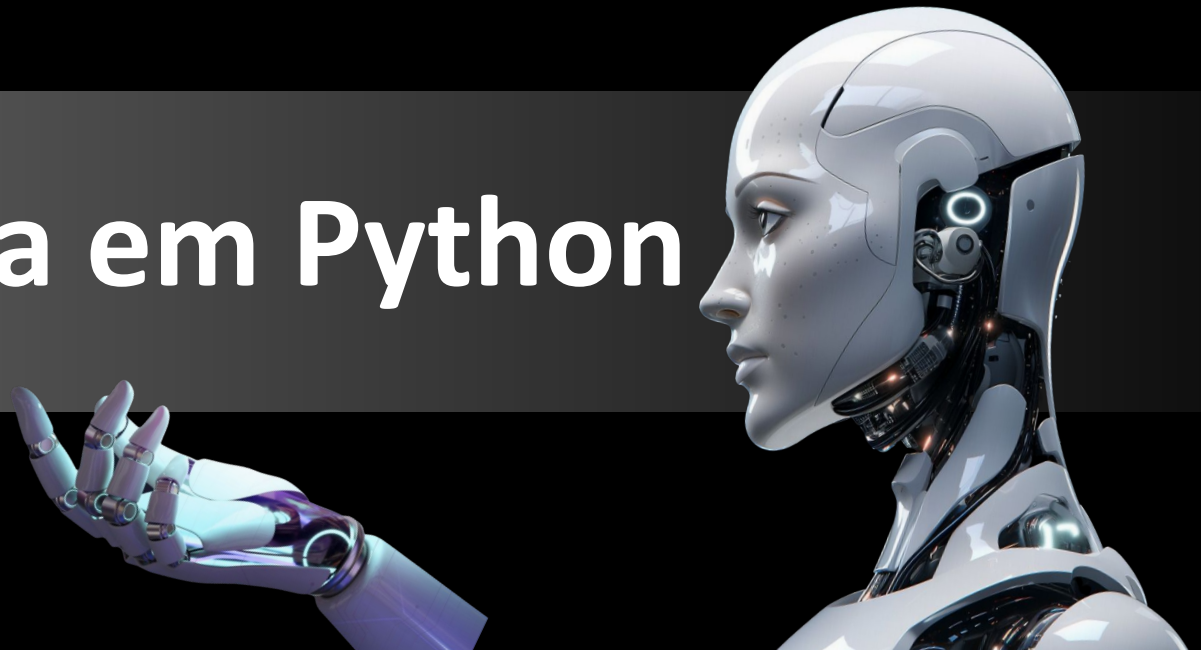
PROJETO EDUCAÇÃO

DO FUTURO

MÓDULO

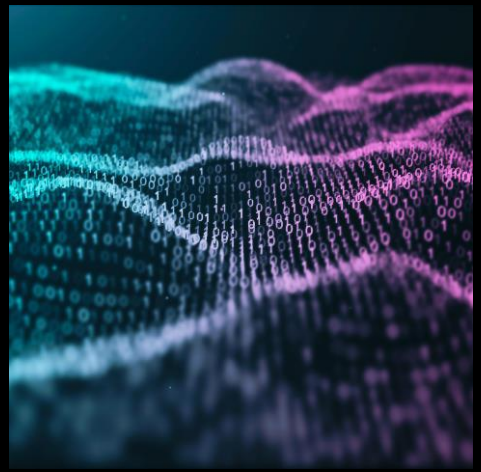
Lógica em Python

AULA 04



Sumário

- Variáveis Simples
- Variáveis Compostas
- Listas
- Desafio



Variáveis Simples

- Uma variável é um espaço reservado na memória do computador onde é possível armazenar um, e somente um único valor, no mesmo instante de tempo.

- Cada uma das variáveis que declaramos em nosso algoritmo pode armazenar um valor e somente um valor. Esse valor pode ser alterado durante a execução do algoritmo de forma que os valores são substituídos, fazendo jus ao nome variável.

- Contudo, a computação surgiu para trabalhar com massa de dados, ou seja, trabalhar com um conjunto grande de dados, e o uso de variáveis simples não colaboram nessa ação. Por isso utilizaremos um tipo diferente de organização de dados: variáveis compostas.

Variáveis Compostas

- A variável composta permite agrupar vários valores em uma única variável.
- Variáveis compostas podem ser classificadas em duas visões:
  - **Variável composta homogênea**: armazenamento de um conjunto de valores de mesmo tipo. Esse tipo de estrutura também pode ser chamado de array unidimensional/multidimensional homogêneo.
  - **Variável composta heterogênea**: armazenamento de um conjunto de valores de diferentes tipos. Esse tipo de estrutura também pode ser chamado de array unidimensional/multidimensional heterogêneo.

Listas

- Declarando uma lista
- Acessando valores de uma lista
- Modificando valores uma lista
- Atividades
- Fatiamento de listas
- Tabela de operadores
- Iterando sobre listas
- Funções internas de listas
- Atividades

Declarando uma lista

- Em Python, listas de valores são representadas pelo tipo list. Esse tipo de dados é basicamente uma sequência de elementos, que podem ou não ser do mesmo tipo.

- Uma lista é um tipo de dado que assim como uma variável deve ser criada e receber algum valor. Porém no caso da lista, um ou mais valores de diversos tipos podem ser passado, para isso utilizamos os colchetes [] e separamos os seus valores por vírgula.

Declarando uma lista

Colchetes utilizado para definir uma lista

```
lista_vazia = []

print(f"Exemplo de lista vazia: {lista_vazia}")
print(f"Tipo de uma lista: {type(lista_vazia)}")
```

Declarando uma lista

Função list() utilizada para definir uma lista

```
lista_vazia = list()

print(f"Exemplo de lista vazia: {lista_vazia}")
print(f"Tipo de uma lista: {type(lista_vazia)}")
```

Declarando uma lista

```python
lista_inteiros = [1, 2, -1, -2]

print(f"Exemplo de lista de inteiros: {lista_inteiros}")
```

Declarando uma lista

```python
lista_tipos_diferentes = ["Python", "nota", 1000, '!']

print(f"Exemplo de lista com tipos diferentes: {lista_tipos_diferentes}")
```

Atividades

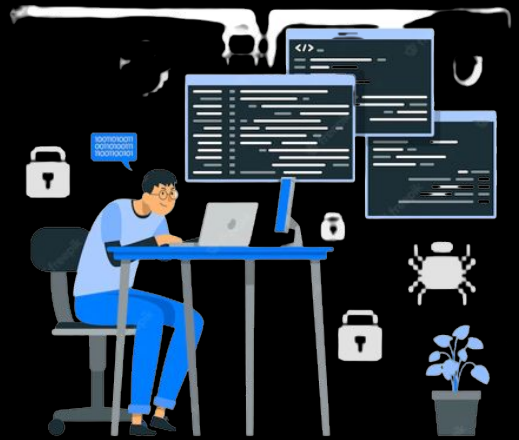
1. Crie uma lista com vários valores e imprima essa lista.

2. Faça uma lista com 5 valores do mesmo tipo e depois exiba-os na tela .

Acessando valores de uma lista

- Como as listas possuem diversos valores, podemos acessar ou modificar o item de uma lista acessando o seu índice.

- O primeiro índice da lista sempre inicia com 0 e termina com a quantidade de itens da lista. Também podemos acessar o último item de uma lista que começa com -1.



Acessando valores de uma lista

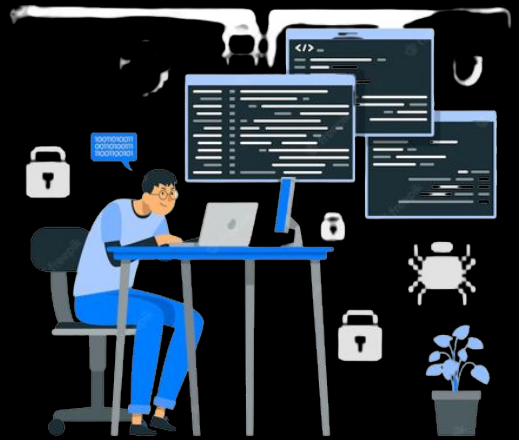
Para acessar, basta usar o identificador da lista, passando o índice entre os colchetes

```
lista_tipos_diferentes = ["Python", "nota", 1000, '!']

print(f"Acessando o primeiro item da lista: {lista_tipos_diferentes[0]}")
```

Modificando valores uma lista

- Para modificar o valor de algum item na lista, basta utilizar o nome da variável e entre colchetes colocar o índice que se deseja acessar e em seguida, atribuir o novo valor, como já é feito em uma variável normalmente.



Modificando valores uma lista

Posição acessada

Valor antigo

```
lista_tipos_diferentes = ["Python", "nota", 1000, '!']

lista_tipos_diferentes[2] = 10000

print(f"Acessando o terceiro item da lista: {
lista_tipos_diferentes[2]}")
```

Novo valor

Atividades

1. Faça uma lista com 5 valores quaisquer e exiba o elemento do índice 0

1. Crie uma lista com valores numéricos e depois some o número do índice 0 e o últimos elemento da lista

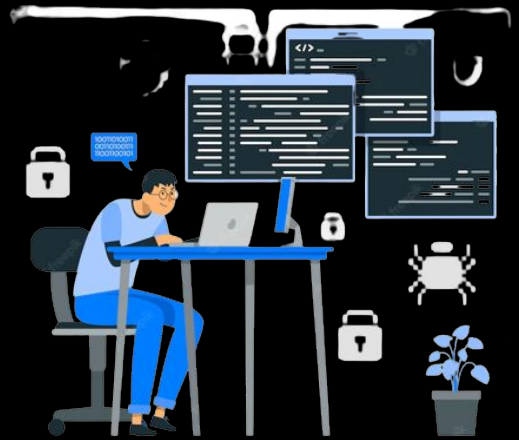
2. Crie uma lista com 5 valores e depois substitua o elemento que está no índice 3.

Fatiamento de listas

- Fatiamento de listas é a extração de um conjunto de elementos contidos numa lista.

- Para extrairmos um conjunto de elementos de uma lista, utilizamos inicialmente, a mesma notação utilizada para a obtenção de um único elemento, porém, ao invés de definirmos somente o elemento que desejamos, informaremos agora, o intervalo desejado.

- O fatiamento de uma lista se dá utilizando [::] onde se define o início, o fim e o seu passo.



Fatiamento de listas

Início

Fim

Passo

```
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Extraindo 5 itens de uma lista
print(f"Exibindo 5 itens: {lista[0:5:1]}")
```

Fatiamento de listas

Início

Fim

Passo

```
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Extraindo 5 itens de uma lista
print(f"Exibindo 5 itens: {lista[5:0:-1]}")
```

Fatiamento de listas

Início

Fim

Passo

```
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Extraindo 5 itens de uma lista
print(f"Exibindo os itens de uma lista, pulando 2 em 2: {lista[0:10:2]}")
```

Tabela de operadores

Operador	Descrição	Exemplo
+	concatena (soma) duas listas	[1, 2, 3] + [4, 5] retorna [1, 2, 3, 4, 5]
*	repete (replica) uma lista n vezes	[0, 1] * 2 retorna [0,1, 0, 1]
[i]	retorna o i-ésimo elemento da lista	[10, 20, 30][0] retorna 10
[i:j]	retorna uma sub-lista que vai dos índices i até j - 1	[1, 1, 2, 3][1:3] retorna [1, 2]

Atividades

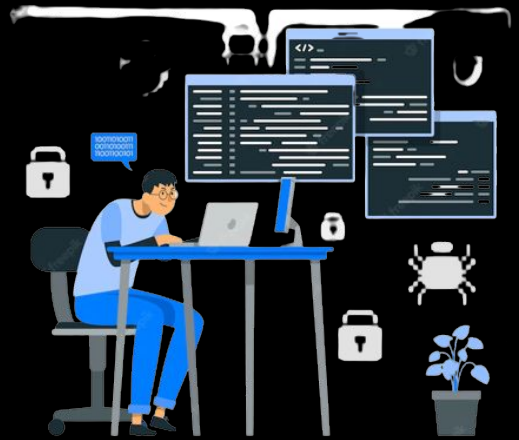
1. Crie uma lista de 0 até 5 e outra lista de 6 até 10, depois some as duas

2. Crie uma lista com 5 valores numéricos e some os três últimos itens da lista.

1. Crie uma lista com 5 valores e multiplique ela por 2

Iterando sobre listas

- Listas são objetos iterável, isso significa que podemos percorrer uma lista utilizando estruturas de repetição.
- A iteração tem o objetivo de percorrer os elementos contidos numa lista. Estudamos, que o Python possui 2 instruções definidas para que sejamos capazes de trabalhar com Iteração, que são, a estrutura while e a estrutura for.



Iterando sobre listas

Valor atual da lista por iteração

Palavra reservada (in) que significa contido em

```
lista = [1, 1, 2, 3, 5, 6, 7, 8, 9, 10]

for item in lista:
  print(f"Valor: {item}")
```

Iterando sobre listas

Função que retorna o tamanho de uma lista

```
lista = [1, 1, 2, 3, 5, 6, 7, 8, 9, 10]

for index in range(0, len(lista)):
  print(f"Valor: {lista[index]}")
```

Atividades

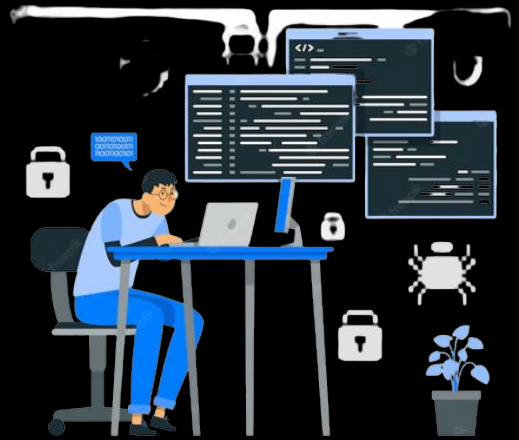
1. Faça uma lista com 10 números e depois percorra cada elemento com um laço de repetição e mostre cada um na tela

2. Faça uma lista com 10 números e depois percorra cada elemento com um laço de repetição, porém exiba somente os números pares.

Funções internas de listas

- No python, toda estrutura possui métodos internos que realizam algumas funções. No python, as listas também usufruem desse benefício.

- Os principais métodos internos das listas são:
  - **append()**
  - **insert()**
  - **pop()**
  - **del()**



Append()

- A função append() irá adicionar um item ao final da lista. O único argumento é o valor a ser inserido.

```
lista = ["Arroz", "Feijão", "Macarrão"]

# Adicionando o item "Carne" ao final da lista
lista.append("Carne")

print(lista)
```

Insert()

- A função insert irá incluir um item na posição que você especificar. O primeiro argumento é índice e o segundo é o valor inserido.

```
lista = ["Arroz", "Feijão", "Macarrão"]

# Adicionando o item "Carne" no índice 0 da lista
lista.insert(0, "Carne")

print(lista)
```

Pop()

- A função pop irá apagar um item da lista. Podemos passar o índice por parâmetro para especificarmos qual item desejamos apagar. Ou podemos apagar o último item da lista, desde que não venhamos a passar nenhum valor.

```python
lista = ["Arroz", "Feijão", "Macarrão"]

# Removendo o primeiro valor da lista
lista.pop(0)

# Removendo o último valor da lista
lista.pop()


print(lista)
```

Del()

- A função del, faz a mesma coisa que a função pop. A diferença é que com a função del, você pode passar um intervalo de valores a serem excluídos, utilizando fatiamento de listas.

```python
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Removendo os valores do índice 2 à 5
del(lista[2:5])

# Removendo o valor do índice 0
del(lista[0])

print(lista)
```

Atividades

1. Faça uma lista vazia e após isso adicione 5 elementos dentro dela utilizando o input.

1. Crie uma lista com 5 elementos e remova os últimos 3 elementos de dentro dela.

DESAFIO

Faça um programa que simule um lançamento de dados. Lance o dado 100 vezes e armazene os resultados em uma lista. Depois, mostre quantas vezes cada valor foi conseguido. Dica: use uma lista de contadores(1-6) e uma função para gerar números aleatórios, simulando os lançamentos dos dados.

Obrigado pela atenção! Até a próxima :)